

Double Ratchet Demonstration

Code Walkthrough & Explanation

This document breaks down the `demo_double_ratchet.py` script part by part, explaining the mechanics of the Double Ratchet algorithm, including Diffie-Hellman ratchets, symmetric ratchets, out-of-order delivery handling, and message authentication.

1. Imports

```
import os
import binascii
import time
```

Why?

- `os` → generate random bytes
- `binascii` → convert bytes to hex
- `time` → pause between demo steps

```
from cryptography.hazmat.primitives.asymmetric import x25519
```

Why?

Used for: `x25519.X25519PrivateKey.generate()` which creates Diffie-Hellman key pairs. Signal uses X25519 for DH key exchange.

```
from double_ratchet import (
    DoubleRatchet,
    DecryptionError,
    header_to_dict,
    header_from_dict,
    bytes_to_b64,
    b64_to_bytes,
    public_key_to_bytes,
)
```

This imports functions from your actual implementation. Think of it like:

```
demo_double_ratchet.py
    ↓
    uses
    ↓
double_ratchet.py
```

The real crypto is inside `double_ratchet.py`.

2. Helper Functions

hexshort()

```
def hexshort(data: bytes, n: int = 16) -> str:
    """Show the first n bytes of data as hex, with an ellipsis if truncated."""
    h = binascii.hexlify(data).decode()
    shown = h[: n * 2]
    suffix = "... " if len(h) > n * 2 else ""
    return f"{shown}{suffix} ({len(data)} bytes)"
```

Purpose: Convert bytes into short readable hex.

Example: `b' 4V'` becomes `123456`. If data is long: `1234567890abcdef...`

section()

```
def section(title: str):  
    print("  
" + "=" * 78)  
    print(f" {title}")  
    print("=" * 78)
```

Prints:

```
=====
PHASE 1
=====
```

Only for formatting output.

step()

```
def step(msg: str):  
    print(f"  
-> {msg}")
```

Prints: -> Alice sends a message

detail()

```
def detail(label: str, value: str):  
    print(f"          {label:<22}: {value}")
```

Prints: RK : 123abc... (Nicely aligned).

pause()

```
def pause(seconds: float = 0.0):  
    if seconds:  
        time.sleep(seconds)
```

Allows animation effect during the demonstration. Currently set to `pause(0)` so nothing happens.

3. dump_state()

This is VERY important as it shows all internal variables of the ratchet state.

```
def dump_state(label: str, rs: DoubleRatchet):  
    """Print a snapshot of one side's full ratchet state."""  
    print(f"  
    [{label}'s internal state]"  
    detail("RK (root key)", hexshort(rs.RK))  
    detail("DHs (our priv->pub)", hexshort(public_key_to_bytes(rs.DHs.public_key())))  
    detail("DHR (their pub)", hexshort(public_key_to_bytes(rs.DHR)) if rs.DHR else  
    "None")  
    detail("CKs (sending chain)", hexshort(rs.CKs) if rs.CKs else "None")  
    detail("CKr (receiving chain)", hexshort(rs.CKr) if rs.CKr else "None")  
    detail("PN / Ns / Nr", f"{rs.PN} / {rs.Ns} / {rs.Nr}")  
    detail("Skipped keys stored", str(len(rs.MKSKIPPED)))
```

Root Key (RK): `detail("RK", hexshort(rs.RK))` - Used to generate future chain keys.

Sending DH Key (DHs): `rs.DHs.public_key()` - Our DH key pair (e.g., Alice's DH public key).

Receiving DH Key (DHr): `rs.DHr` - Stores the other person's public key (e.g., Alice stores Bob's key).

Sending Chain Key (CKs): `rs.CKs` - Used to generate message keys (MK1, MK2, MK3...).

Receiving Chain Key (CKr): `rs.CKr` - Used to decrypt incoming messages.

Counters:

Variable	Meaning
<code>PN</code>	Previous chain length
<code>Ns</code>	Sent messages count
<code>Nr</code>	Received messages count

MKSKIPPED: `len(rs.MKSKIPPED)` - Stores skipped message keys. Needed for out-of-order delivery.

4. send_message()

This function demonstrates a full send/receive cycle.

```
def send_message(sender_label, sender, receiver_label, receiver, plaintext: bytes,
ad: bytes = b''):
    ...
    # Sender Encrypts
    header, ciphertext = sender.encrypt(plaintext, ad)

    # Simulate network transport: serialize to JSON-safe forms and back.
    wire_header = header_to_dict(header)
    wire_ciphertext = bytes_to_b64(ciphertext)

    received_header = header_from_dict(wire_header)
    received_ciphertext = b64_to_bytes(wire_ciphertext)

    # Receiver Side
    dhr_known = public_key_to_bytes(receiver.DHr) if receiver.DHr else None
    new_key_arrived = received_header["dh_pub"] != dhr_known

    plaintext_out = receiver.decrypt(received_header, received_ciphertext, ad)
    ...
```

Sender Encrypts: `header, ciphertext = sender.encrypt(plaintext, ad)`

Inside `encrypt()`: CKs → KDF → Message Key → AES-GCM → Ciphertext.

Header Contains:

- `dh_pub`: Current DH public key.
- `pn`: Previous chain length.
- `n`: Message number.

Serialization: Messages must travel through the network. Convert header: `wire_header = header_to_dict(header)` and convert ciphertext: `wire_ciphertext = bytes_to_b64(ciphertext)` because JSON can't store raw bytes.

Receiver Side: Convert back into usable form using `header_from_dict(...)`. Check for new DH key: `new_key_arrived = received_header["dh_pub"] != dhr_known`. If different, a New Ratchet Step occurs. If the same, it remains on the same chain.

Finally, **Decrypt:** `plaintext_out = receiver.decrypt(...)`. The receiver derives the message key, decrypts, and verifies the authentication tag.

5. MAIN FUNCTION

Now the actual demonstration begins, structured into phases.

PHASE 0: Initializing Secrets

```
SK = os.urandom(32)
bob_spk_private = x25519.X25519PrivateKey.generate()
bob_spk_public = bob_spk_private.public_key()
```

Creates the **Shared Secret** (pretending it came from X3DH).

`bob_spk_private` and `bob_spk_public` represent Bob's Signed PreKey pair.

In reality: X3DH → SK → Double Ratchet starts.

PHASE 1: Session Initialization

```
alice = DoubleRatchet(SK, bob_spk_public, is_bob=False)
bob = DoubleRatchet(SK, bob_spk_private, is_bob=True)
```

Alice: is the initiator. She immediately performs `DH(Alice, BobSPK)` and derives `RK` and `CKs`.

Bob: waits. He has no sending chain yet (`CKs = None`).

PHASE 2: Alice Sends First Message

```
send_message("Alice", alice, "Bob", bob, b"Hi Bob...")
```

Bob receives the message. The header contains Alice's DH public key. Bob sees a **New DH key!** and performs a **DH Ratchet**, generating a new `RK'` and `CKr`.

PHASE 3: Bob Replies

```
send_message("Bob", bob, "Alice", alice, b"Hey Alice!...")
```

Now Bob generates his own ratchet key pair. Alice receives it and notices Bob's DH key changed. Another DH Ratchet occurs. Both sides are now fully initialized with `RK`, `CKs`, and `CKr`.

PHASE 4: Symmetric Ratchet

```
for i in range(3):
    send_message("Alice", alice, "Bob", bob, ...)
```

Alice sends 3 messages without replies. No DH Ratchet occurs. Only the **Symmetric Ratchet** operates: `CKs → KDF_CK → new CKs`.

PHASE 5: Out-of-Order Delivery

```
h_first, c_first = alice.encrypt(b"FIRST")
h_second, c_second = alice.encrypt(b"SECOND")

out2 = bob.decrypt(h_second, c_second) # Delivered first
out1 = bob.decrypt(h_first, c_first)   # Delivered second
```

Network delivers #2 first, and #1 second. When Bob receives #2, he realizes Message #1 is missing. He stores the key for message #1 in `MKSKIPPED`.

When #1 arrives later: Use saved key → Decrypt → Delete saved key. (Exactly how Signal handles delayed packets).

PHASE 6: Tampering Attack

```
h, c = alice.encrypt(b"Message")
tampered = bytearray(c)
tampered[-1] ^= 0xFF
```

Alice encrypts. We flip one bit in the ciphertext. When Bob decrypts, the AES-GCM checks the authentication tag, which fails, raising a `DecryptionError`. The message is rejected, proving **Integrity** and **Authentication**.

PHASE 7: Session Persistence

```
state = alice.to_dict()
alice = DoubleRatchet.from_dict(state)
```

Very important for a Flask project. Serializes the internal state to a dictionary for DB storage. Rebuilding the state from the dict simulates a server restart. Without this, every restart would destroy the session.

PHASE 8: Security Guarantees

- **Forward Secrecy:** Each message gets a new Message Key. Stealing today's key cannot decrypt yesterday's messages.
- **Future Secrecy (Self-Healing):** Every DH Ratchet creates a New DH Key Pair → New Root Key → New Chains. A compromised chain eventually heals.
- **Out-of-Order Resilience:** Handled using `MKSKIPPED`.
- **Authentication:** Handled by AES-256-GCM Tag. If even 1 bit changes, the message is rejected.

Overall Flow:

X3DH → Shared Secret (SK) → Double Ratchet Init → Alice Sends → Bob DH Ratchet → Bob Replies → Alice DH Ratchet → Symmetric Ratchet Messages → Out-of-Order Handling → Tamper Detection → Save/Restore Session